ENGINEERING
W Booth School of Engineering
Practice and Technology

McMaster University

SEP 780 – Advanced Robotics and Automation

Final Project Report

A Vision-Enabled Smart Robot Car

Date: April 3, 2026

Group 2:

Member Name	Student Number
Tianyu Zhang	400673446
Zhuofan Sun	400359854
Jingxuan Li	400365136
Hongyan Chang	400679128

Table of Contents

1 Introduction and Background 2

2 Detailed Implementation 2

2.1 Hardware Assembly and Core Functionality Validation 2

2.2 Advanced Feature 1: X-BOX Controller Integration 6

2.3 Advanced Feature 2: Camera-Based Object Detection and PC Communication 8

2.4 Project Timeline and Team Contribution 10

3 Results and Demonstration 11

4 Challenges and Solutions 13

5 Lessons Learned 14

6 Conclusion 15

7 References 16

1 Introduction and Background

The main objective of the project was to gain hands-on experience in robotics, electronics, and programming through the use of the ELEGOO Robot Car Kit V4.0, which is an Arduino-based educational platform and provides a robust hardware foundation and many pre-written code examples for basic operations. The initial proposal focused on assembling the kit and validating the core functionalities, including IR remote control, line tracking, obstacle avoidance, and automatic following. Additional extensions such as programmed movement sequences and RGB LED signaling were considered as optional follow-up work. After the car assembly and baseline validation were completed successfully, the project scope was expanded from validating stock examples to building a custom multi-component robot system. In the final implementation, the Arduino UNO remained responsible for execution and safety, the ESP32-S3 handled camera, Wi-Fi, UI hosting, and serial bridging, an Angular web application became the operator console, and a local Python backend was added for computer vision. Two major visible outcomes were therefore a browser-based control pipeline centered on the X-BOX controller and a real-time vision-assisted workflow between the robot and a host computer.

2 Detailed Implementation

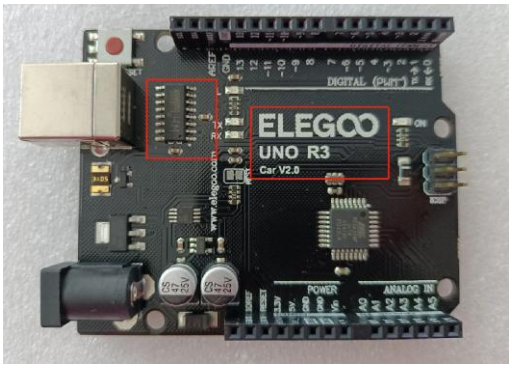
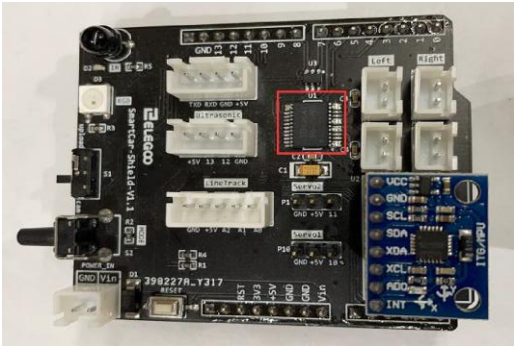
2.1 Hardware Assembly and Core Functionality Validation

At the start of the project, we assembled the ELEGOO Robot Car Kit V4.0 following the manual from the manufacturer. The process includes the mounting of motors, line-tracking sensors, battery module, ultrasonic sensor, camera module, the main control board (Arduino UNO R3) and other mechanical parts.

The user manual and example projects were used as the baseline references during the beginning phase, but after getting familiar with the robot car, the final control, communication, and vision software were progressively replaced by custom modules.

Table 1 lists the main hardware components that were used in the project, including the Arduino main control board, motor driver board, IR sensor, ultrasonic sensor, etc. Please refer to the table for details.

Table 1. The main hardware components

Number	Description	Photo
1	Arduino UNO R3: ATmega328P microcontroller CH340 USB to Serial chip	
2	IO expansion board: TB6612 motor drive chip MPU6050 gyro module	
3	Ultrasonic sensor module: HC-SR04	
4	Line tracking module ITR20001	
5	Camera module: ESP32-S3-WROOM-1	

Number	Description	Photo
6	SG90 Servo motor	
7	DC Motor	

The fully assembled robot car as below:

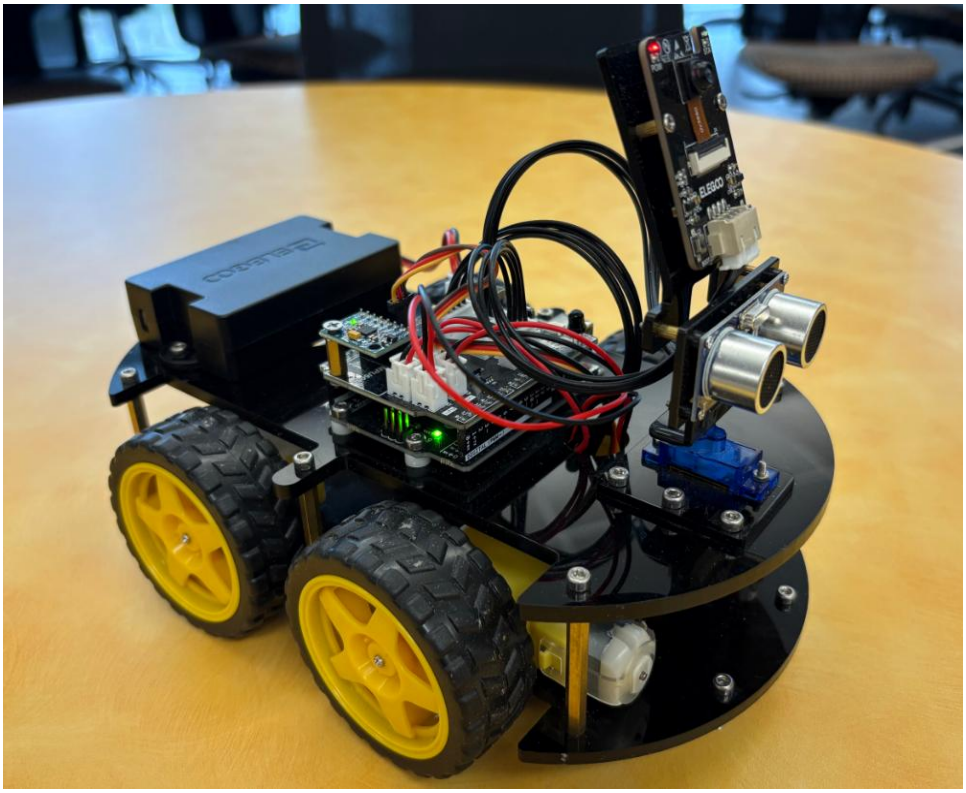


Figure 1. The fully assembled ELEGOO Robot Car

After the robot car was fully assembled, all the core functionalities were validated.

IR Remote Control. Basic driving such as forward, backward, left turn and right turn with the remote. All the basic directional control was tested successfully.

Line Tacking. The robot car has the ability to follow a black line on a white surface using three infrared sensors. Also tried to use some other line to simulate the black line, but it did not work. High contrast is required.

Obstacle Avoidance. The car can move in a room autonomously by using the HC-SR04 ultrasonic sensor to detect and avoid obstacles by turning left or right.

Automatic Following. The car can detect the objects ahead by measuring the distance to objects continuously and following the target. It also uses the ultrasonic sensor to detect the distance.

The control logic for each function is handled by the main control board (Arduino UNO R3), which acts as the control center. Corresponding sensors, such as infrared sensors and ultrasonic sensors, sense information from the external environment and transmit it to the main control board. The main control board controls the actuators (motors) to perform a series of actions based on the information, such as moving, line tracking, obstacle avoidance, and automatic following. The overall system architecture is shown in Figure 2.

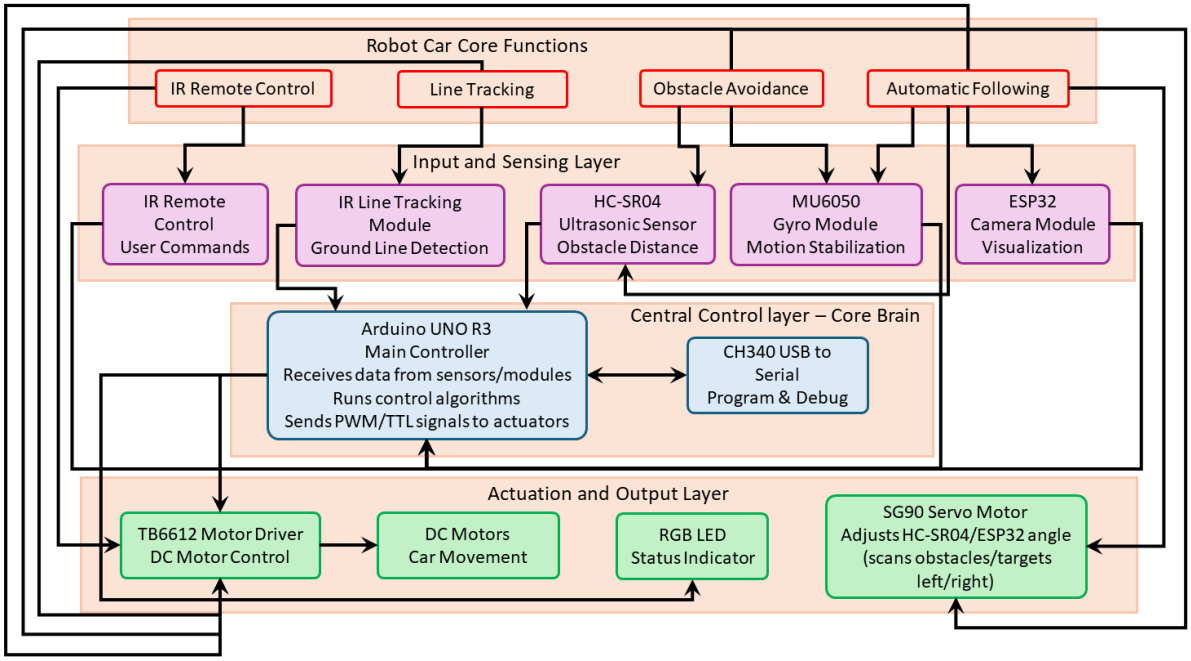


Figure 2. The system architecture diagram of the Robot Car

2.2 Advanced Feature 1: X-BOX Controller Integration

The goal of this feature was to achieve a more immersive and precise control experience through the use of an X-BOX game controller. In practice, this evolved into a browser-based teleoperation pipeline in which the controller was the main demonstration input device, while the same frontend pathway was later extended to keyboard control and drive-mode management.

User Interface (UI). An Angular-based web application is developed to act as a central hub, capturing controller and keyboard input events, displaying robot state, and serving as the operator dashboard.

Figure 3. The UI Demo Example

Communication Protocol. WebSocket technology is employed for its low-latency, full-duplex communication capabilities. The Angular UI functions as a WebSocket client, can transmit the control commands to the robot and receive state updates in return.

Robot Endpoint. The ESP32-S3 camera module of the robot is configured as the robot-side HTTP and WebSocket endpoint, typically operating in Access Point mode during demonstration. It also serves the static UI files and MJPEG video stream.

Microcontroller Interface. The ESP32-S3 forwards motion and servo commands to the Arduino UNO via a serial connection. The UNO then translates these high-level commands into specific motor control signals and applies local safety logic before driving the motor layer.

Figure 4 is the architecture diagram with an integrating X-BOX controller and Figure 5 shows the keyboard of X-BOX controller.

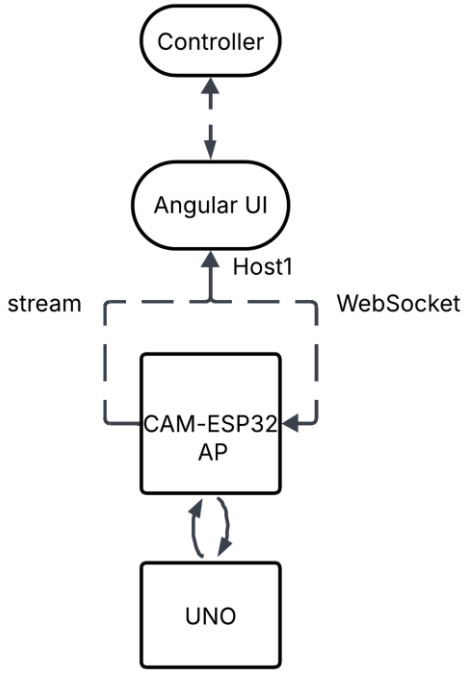


Figure 4. The architecture diagram of integrating X-BOX controller

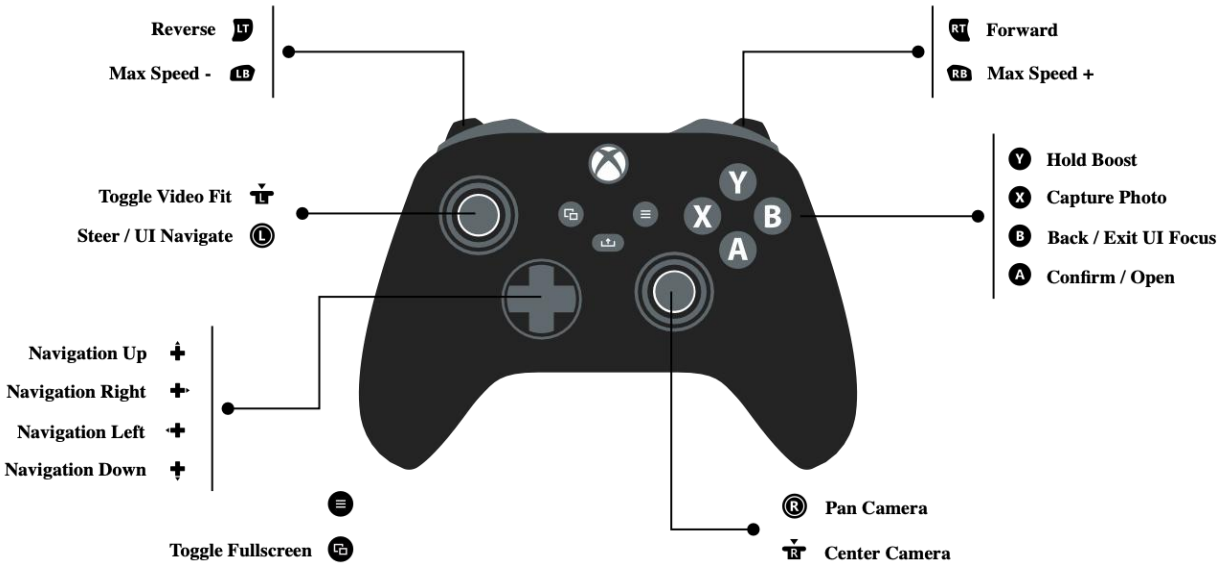


Figure 5. X-BOX controller

2.3 Advanced Feature 2: Camera-Based Object Detection and PC Communication

The goal of this feature was to equip the robot with visual perception and allow vision results to be processed and displayed on a host computer in real time.

Image Capture. The ESP32-S3 camera module on the robot continuously captures video frames.

Video Streaming. The ESP32-S3 streams the live video feed over Wi-Fi to the Angular UI on the host computer via a dedicated channel.

Object Detection. Two processing directions were considered during development, but the final implementation adopted the backend-CV path.

Frontend CV. Browser-side processing was considered during early design because it simplified deployment, but it was not retained as the final solution.

Backend CV. In the implemented system, the Angular UI relays selected JPEG frames to a local Python backend via WebSocket. The backend uses YOLO26l and ByteTrack to detect and track a person, then returns normalized detections and target-lock state to the UI.

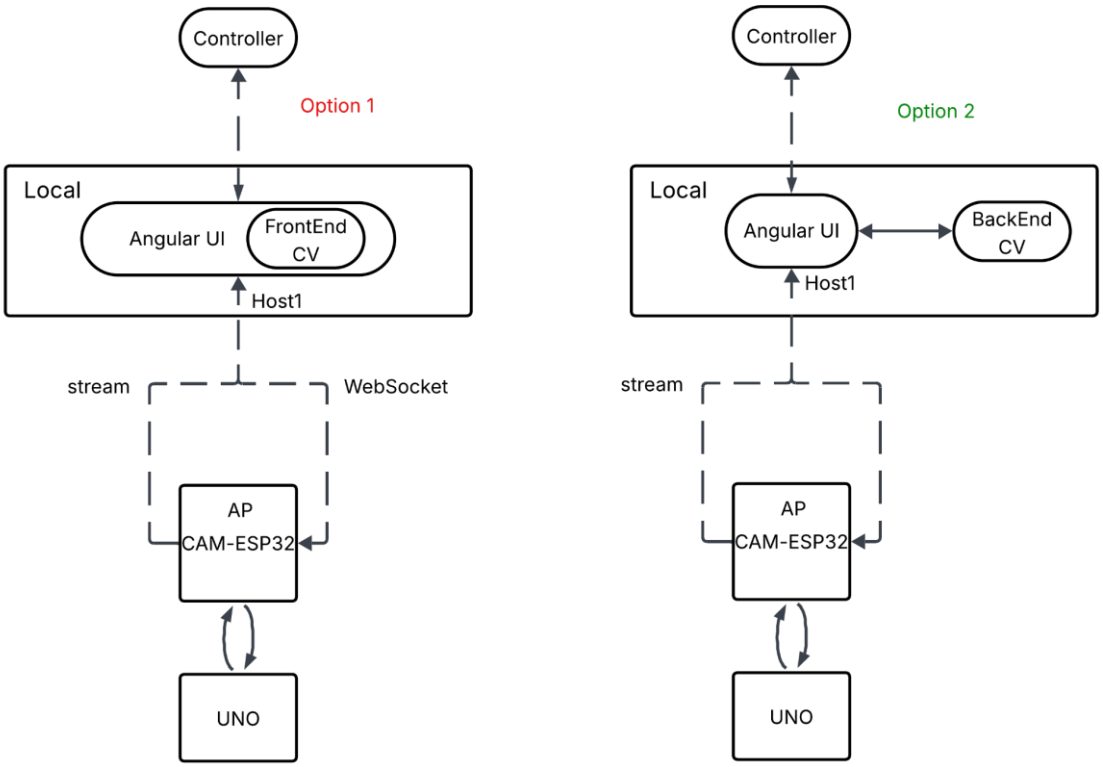


Figure 6. The architecture diagram of camera-based object detection and PC communication

Data Visualization. The Angular UI displays the live video feed and overlays detection results on the screen. The returned target information is also used to support `manual`, `assist`, and `auto` drive modes, while visual alerts notify the user in real time upon identification of a target object.

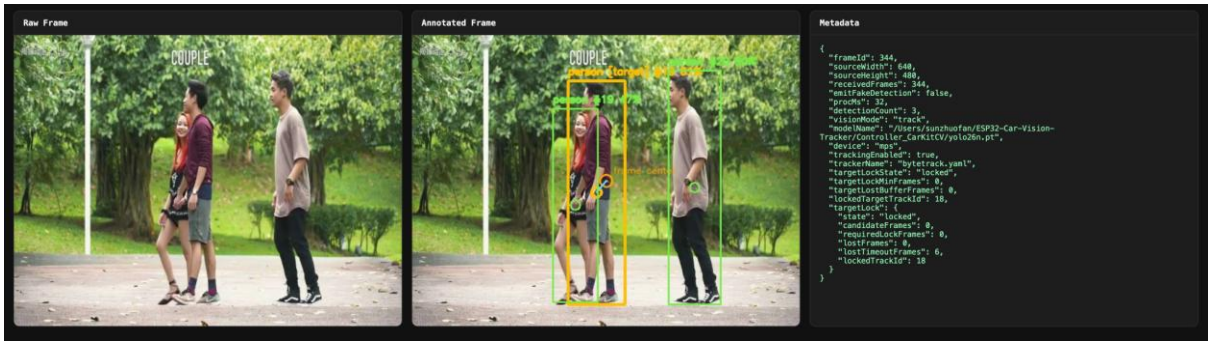


Figure 7. Backend Debugging Snapshot

2.4 Project Timeline and Team Contribution

The project was completed on schedule with clear task division and strong collaboration. The major milestones of the project were the main function test and validation, additional features development, test and validation. The additional capabilities were successfully implemented, including browser-based teleoperation, host-side computer vision integration, and later stabilization and UI refinement. A dual-ESP32 hardware branch was also explored briefly before the architecture converged back to the simpler single-ESP32-S3 design. Hardware assembly and main functions (IR control, line tracking, obstacle avoidance, automatic following) were finished ahead of the original plan. Advanced modules including X-BOX handle control, real-time vision streaming, and PC communication were fully integrated and validated for the final demonstration. Each team member delivered clear contributions according to the timing plan.

SEP 780 Project: Automous Robot Car with ELEGOO Robot Car Kit V4.0

Group #2 Team member:
Tianyu Zhang, Zhuofan Sun, Jingxuan Li, Hongyan Chang

Project Start: 2026/1/19
Show Weeks: 7

Task Distribution & Description	Responsible Name	Progress	Start Date	End Date	Status
Project Familiar & Purchase Kit					
Learning knowledge of Mobile Robotics	All Team members	100%	2026/1/19	2026/2/2	
Learning knowledge of Arduino IDE (code)	All Team members	100%	2026/1/19	2026/2/2	
Compare different Car kits & Purchase Kit	All Team members	100%	2026/2/2	2026/2/9	
Assembly Robot Car Kit	All Team members	100%	2026/2/9	2026/2/22	
Brain-storming for the additional functions	All Team members	100%	2026/2/9	2026/2/22	
Prepare Project Proposal Report	All Team members	100%	2026/2/9	2026/2/22	
Main Function Test & Validation					
IR Remote Control	Tianyu, Zhuofan	100%	2026/2/23	2026/3/9	
Line Tracking	Jingxuan, Hongyan	100%	2026/2/23	2026/3/9	
Obstacle Avoidance	Jingxuan, Hongyan	100%	2026/2/23	2026/3/9	
Automatic Following	All Team members	100%	2026/3/9	2026/3/16	
Prepare Project progress report	All Team members	100%	2026/3/11	2026/3/16	
Additional Function Test & Validation					
Control Robot with X-BOX handle	Tianyu, Hongyan	100%	2026/3/11	2026/3/25	
Smart Car detect objects with camera	Jingxuan, Zhuofan	100%	2026/3/11	2026/3/25	
Final test and validation	All Team members	100%	2026/3/26	2026/3/30	
Project Demo, Presentation & Final Report					
Prepare Project Demo and Presentation	All Team members	100%	2026/3/30	2026/4/1	
Prepare Project Final Report	All Team members	100%	2026/3/30	2026/4/4	

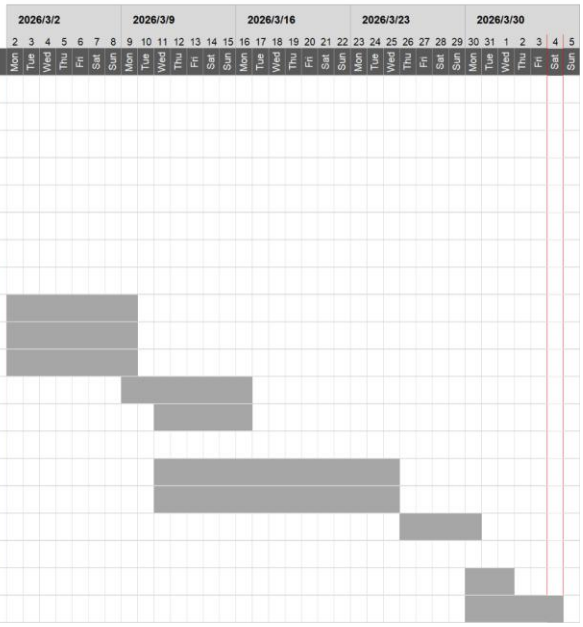


Figure 8. Project timeline and contribution

3 Results and Demonstration

The system integrated the advanced features successfully.

X-BOX & Keyboard Control. Through the browser-based operator console, the system responds to the X-BOX controller analog sticks smoothly for proportional speed and steering control. Keyboard control was also implemented as a testing and fallback input path.

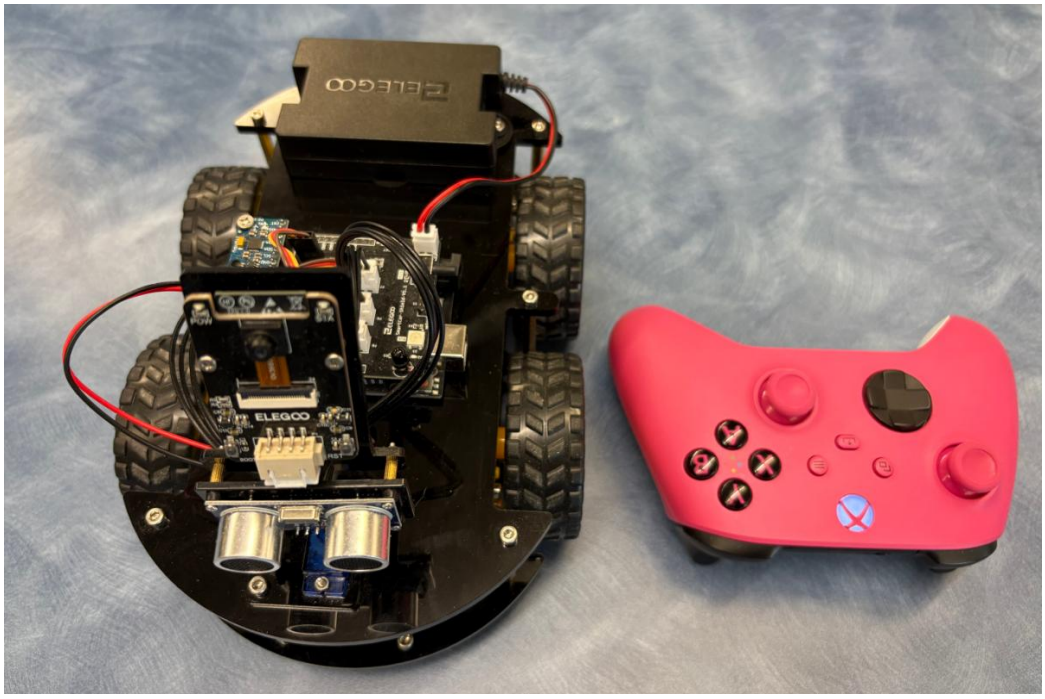


Figure 9. X-BOX controller with the Robot Car

Vision and Communication. The live video streaming from the robot to the computer dashboard was established. Selected frames are relayed from the UI to the local Python CV backend, and the returned detection results are displayed in real time on the dashboard. In assist mode the UI previews steering and throttle suggestions, and in auto mode those suggestions can be applied with manual override and radar-based safety constraints.

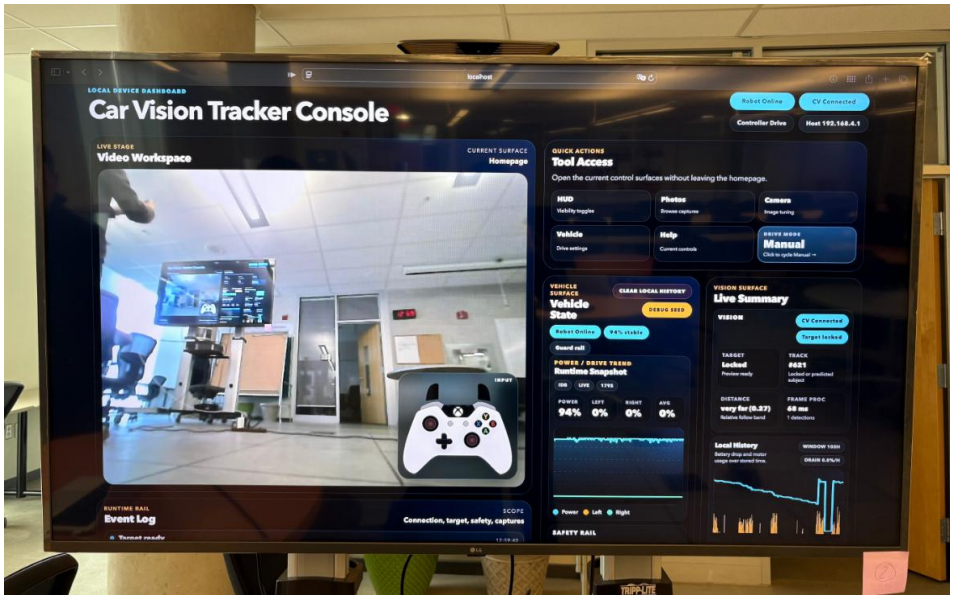


Figure 10. The live video dashboard

Operator Telemetry and Local Logging. To improve observability during testing, the operator UI records both short-window runtime snapshots and long-window local history. Figure 11 (Left) shows the estimated battery level together with left/right drive activity during an active maneuver, while Figure 11 (Right) shows the longer-term battery decline and burst-like drive usage over a 2.3-hour test window. These views were used to validate command output behavior, compare tuning changes, and identify battery-dependent motion variation during extended operation. These plots are operator-side telemetry derived from reported battery voltage and issued drive commands, rather than direct current measurements.

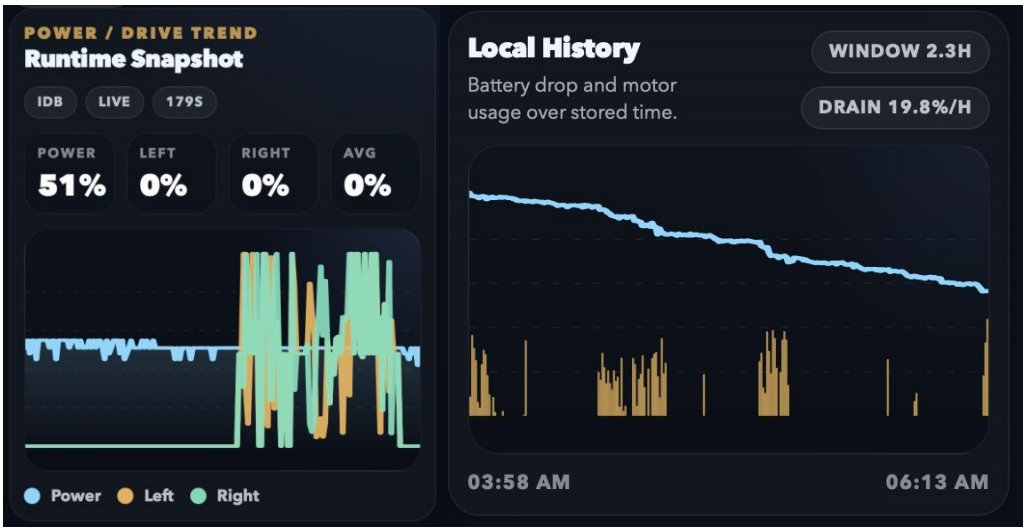


Figure 11. Left) The estimated battery level. Right) Long-term battery & motor history

4 Challenges and Solutions

Several significant challenges were presented while the additional features were being developed.

Wireless Networking and Architecture Trade-offs. Network mode selection had a direct effect on demonstration quality. During development, an additional ESP32 branch was also explored so that one board could manage the access point and local interaction while the ESP32-CAM focused on video. Although this improved video behavior in some tests, the added power demand, unreliable jumper-wire connections, and frequent corrupted frames under battery operation made the branch too fragile. The final system therefore retained the simpler single-ESP32-S3 architecture, with SoftAP mode used as the default demonstration configuration.

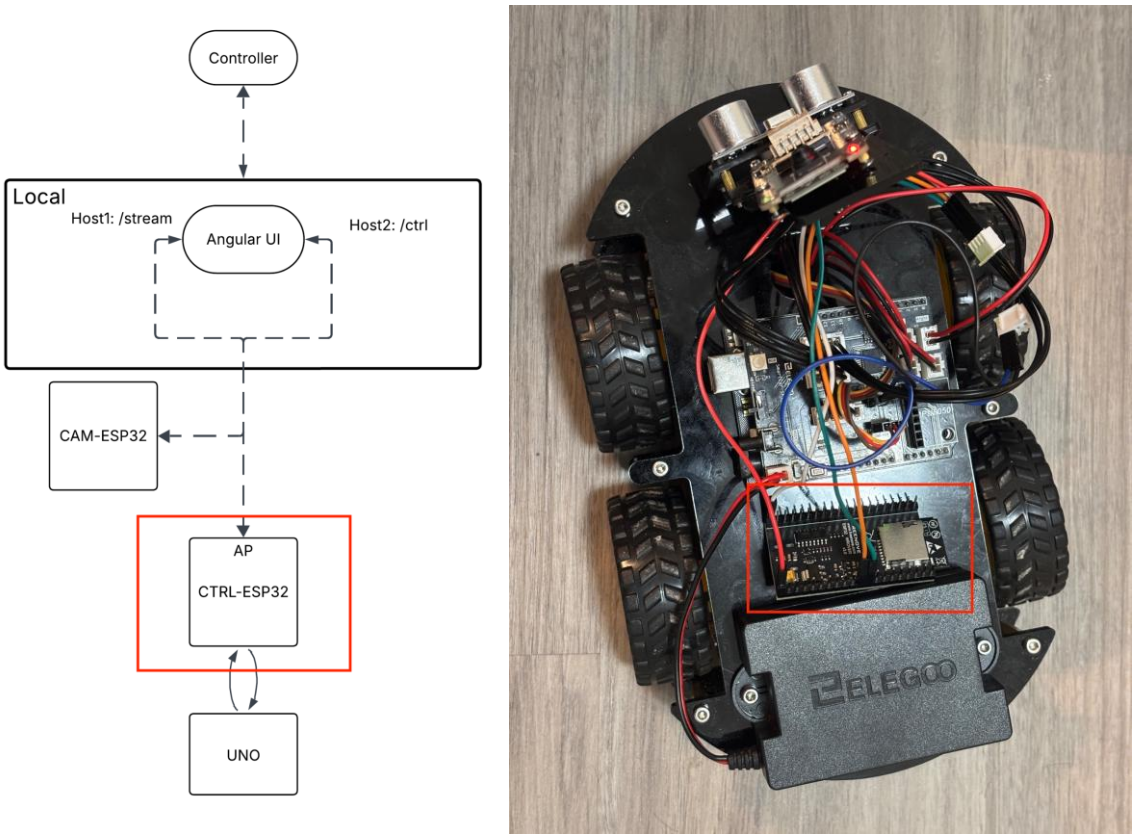


Figure 12. Left) Experimental dual-ESP32 control and streaming concept. Right) Prototype hardware layout of the dismissed dual-ESP32 branch.

Camera Resolution and Relay Latency. The limited resolution and bandwidth of the ESP32 camera made it difficult to detect small or distant objects. Additional latency was also

introduced by browser display, frame relay, JPEG encoding, and backend inference. The system was therefore tuned by balancing stream resolution, relay frame size, JPEG quality, and frame rate. After system tuning, the end-to-end streaming latency was reduced to below 200 ms on average. The backend inference time per frame was maintained under 70 ms.

Serial Communication Stability and Safety. Initial attempts at communication between the ESP32 and Arduino UNO suffered from data corruption, lag, and synchronization problems. This issue was resolved through a more robust serial JSON protocol, incremental state synchronization, and local safety logic on the UNO, including automatic stop when serial input is lost.

Target Detection Robustness and Target-Loss Handling. Reliable tracking required repeated adjustment of thresholds, relative-distance estimation, and target selection logic. The development process therefore moved from fake detections for overlay validation to real YOLO26l plus ByteTrack inference, then later added bounding-box smoothing, short prediction windows, and recovery behavior to reduce false loss of target.

Motion Stability and Power Variation. Vehicle behavior was affected by steering aggressiveness, battery condition, and ultrasonic safety triggering. Later revisions added steering caps at high throttle, more stable radar-based forward safety, and voltage-based speed strategies to maintain more predictable driving behavior. To simplify the final motion stack and control firmware size, previously explored MPU6050-based logic and most stock IR behaviors were not retained in the final custom system.

5 Lessons Learned

The Significance of Hardware Debugging. Ensuring correct wiring and proper sensor function is critical. It often takes longer than expected, so patience and methodical testing are essential.

The Interdependence of Software and Hardware. System performance is highly dependent on the seamless interaction between software logic and hardware components. Modifications in one domain can have significant impacts on the other.

The Value of Modular Design. By separating control, communication, and vision components, a more robust and maintainable system was created. This modularity simplified the debugging of individual parts and facilitated design iteration.

The Value of Incremental Validation. Bringing up the protocol and overlay path with fake detections before switching to real inference greatly reduced debugging complexity and made later integration with the real CV backend more manageable.

Teamwork is important. Complex problems can be solved more efficiently through collaboration, brainstorming and assigning tasks based on individual strengths. Regular communication is considered as a key factor in keeping the project on tack and aligned.

6 Conclusion

In conclusion, the project was developed from the initial kit-based concept into a browser-operated robot car with real-time video, host-side vision processing, and layered control. Beyond validating the core functionality of the ELEGOO Car Kit, the project developed and implemented a customized multi-component system in which the Arduino UNO handled execution and safety, the ESP32-S3 handled camera, network, and serial bridging, the Angular UI served as the operator console, and the local Python backend provided computer-vision support. This project not only deepened the team's understanding of embedded systems and advanced robotics, but also enhanced hands-on skills in networked robot control and real-time visualization.

7 References

ELEGOO Smart Robot Car Kit V4.0 Tutorial

https://www.elegoo.com/blogs/arduino-projects/elegoo-smart-robot-car-kit-v4-0-tutorial

Ultralytics YOLO Documentation

https://docs.ultralytics.com/models/yolo26/

ByteTrack GitHub Repository

https://github.com/FoundationVision/ByteTrack

RobotCarUI GitHub Repository

https://github.com/Thibdumont/RobotCarUI

espCam2 GitHub Repository

https://github.com/Thibdumont/espCam2

RobotCar2 GitHub Repository

https://github.com/Thibdumont/RobotCar2

MDN Gamepad API

https://developer.mozilla.org/en-US/docs/Web/API/Gamepad_API